

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHASSIST
13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

Table of Contents

Abstract	2
1 Introduction	5
1.1 Problem statement	5
1.2 Objectives of the Project	5
1.3 Methodology Overview	5
1.4 Contributions	5
2 Background and State of the Art	6
2.1 Dynamic biped locomotion	6
2.2 Control approaches	6
2.3 RL for locomotion	6
2.4 Existing robot platforms	6
3 Development Methodology	7
3.1 Overall methodology	7
3.2 Hardware selection	7
3.2.1 Motor	7
3.2.2 Battery pack	7
3.2.3 Compute	7
3.2.4 Sensors	8
3.3 Robot modelling	8
3.4 Simulation	8
3.5 Achievable gait: actuator and mass-distribution limits	8
3.5.1 Method	8
3.5.2 Results	9
3.5.3 Limiting factors	9
3.5.4 Sensitivity to mass distribution	10
3.5.5 Ankle stiffness	10
3.5.6 Limitations	10
3.6 Toy problem / RL Warmup	10
3.7 RL policy design and validation	10
3.7.1 Attempt 1: direct joint torque	11
3.7.2 Attempt 2: joint position targets (PD)	11
3.7.3 Attempt 3: a central pattern generator tuned by the policy	12
3.7.4 Reflexes: what is learned and what is hard-wired	14
3.7.5 The pitch-balance wall, and how it was broken	16
3.7.6 The cadence problem and a cautionary bug	17
3.7.7 Results	18
3.7.8 Validation status and known gaps	18
3.8 deployment	19
3.8.1 linux and RT	19
3.8.2 CAN	19
3.8.3 motor controller	19
3.8.4 hardware validation	19
3.9 Hardware experiments	19
4 Results	20

4.1	Hardware selection	20
4.2	RL experiments	20
5	Discussion	21
5.1	Sucesses	21
5.2	Failures	21
5.3	Lessons learned	21
6	Conclusion and Future Work	22
6.1	Summary of Findings	22
6.2	Challenges and Limitations	22
6.3	Suggestions for Future Research	22
7	Appendix	23
7.1	logbook	23
7.2	URDF from CAD:step by step	23
	References	24

1 Introduction

This thesis investigates the practical challenges of applying existing RL methods to a custom high-speed biped robot and provides a reproducible development methodology covering robot modelling, simulation, control integration, and experimental validation.

1.1 Problem statement

1.2 Objectives of the Project

1.3 Methodology Overview

What tool / technic, ... did I use / applied my strategy to solve the problem

1.4 Contributions

What I did, recap of my original work.

What I did : actuator mapping database with metric for figure of merit battery mapping database with metric for figure of merit URDF modelling from scratch linux, CAN, cubemars setup Battery pack design and performance RL toy problem -¿ Spinoff as TP ? RL for running robot, incremental approach and test -¿ torque control -¿ no convergence, too slow with RT linux -¿ Position control -¿ converged before update mass and inertia, not converging anymore... -¿ gait generator approach, fourrier coefficient, simplified problem with reduced DOF, hypothesis that the abduction alone can stay oin position control and be enough to act as reflex system to keep robot stable. running with the based attach on a threadmill. -¿ attempt to train for fully unthethered operation (not done yet)

things to develop

map motor power density how to URDF from CAD correctly. (manual process)

state of the art running robot mechanical (model (SLIP, oscillator,) , timming estimation, ... state of the art RL and bipedal + sim2real sensor choice BOM development electrical system (and control issue + timming and jitter) toy problem to test the system and setup manual for student TP real implementation and training real deployment and challenge

2 Background and State of the Art

Hype in running robot, recent achievement, ...

2.1 Dynamic biped locomotion

the models that exist, why its hard, how to stabilize

2.2 Control approaches

classical vs RL, why RL is interesting

2.3 RL for locomotion

PPO action spaces reward design sim-to-real and all other things I tried / read (fourier, torque vs position, ...)

2.4 Existing robot platforms

im not sure I care much about it. done already by max anyway...

3 Development Methodology

3.1 Overall methodology

hybrid start with robot partially designed, Robot modelling and help with hardware selection for non pure-mechanical part URDF export Robot workspace Validation and calibration Robot dynamical validation against inertia matrix and URDF Simulation and fidelity implementation Toy problem for RL complete loop with one actuator, one linux on BBB, CAN bus, ... to gain experience, RL training and validation (different approach tried) deployment consideration (linux and RT/semi RT kernel), CAN bus bandwidth and refresh speed, motor control mode, hardware validation, back driving and play back,

3.2 Hardware selection

3.2.1 Motor

3.2.2 Battery pack

having select the motor, we knew the current that would be needed to drive them. (

We need to optimise for power density as the robot run are quite short (after all, it took usain bolt less than 10 second to run 100m, I hope our robot can do it in less than 30) Its not a marathon running robot but a pure speed sprinter one.

maybe highlight mismatch between commercial offering and theoretical performance by doing a map like for the motors.

Thus I went one to review cell and chemistry for power density. And while LFP and LTO along with Lipo should be more power dense, some challenge in sourcing theses cell made them an unviable option. Commercial LTO cell where found on ALiexpress but decided as not worth the risk. I could not find any reputable / specialized suppliers that would ship cells. Nkon is the the supplier I have used in the past for battery need and from there I looked at the most power dense battery cell.

Lipo in commercial package could not be found at 48V...

Cost was also a factor of choice

The EVE INR21700-50PL 5000mAh - 125A (nieuwe versie) where selected as they where the most power dense cell we could find.

current pack 13S2P, 200 A BMS (limited thermally, peak limited to 250A in software)

3.2.3 Compute

Most of the lab exoskeleton run on beaglebone black While theses support CAN natively, and run linux, the SSH connection was VERY slow and thus was rejected. The main selling point was the real time coprocessor (understand a beefy MCU that run alongside the main CPU for time critical application) but this require custom code and the performance benefit are still to demonstrate.

A raspberry pi 2 b+ that was laying around was used instead as the author had much more experience with theses. It did require to buy a CAN hat over SPI. another hat with an accelerometer was bought (raspberry pi sense hat) for the ease of integration.

mention here or later the need to check the mapping when stacking the two hats.

Need to document the test for loop frequency and jitter but main result is that we can have a can loop at 200 Hz but and that the raspberry pi is MUCH more powerful than the beagle bone. which is very convenient for development (we host a wifi hotspot so we can connect to the robot, we host a web interface to control the robot to simplify the manipulation)

3.2.4 Sensors

sense hat for base IMU, contact sens but never tried. literature suggested that impact timing and direction can improve RL convergence.

3.3 Robot modelling

Re assembled the robot in autodesk inventor (for fluency reason) and exported as STEP each limb with the origin centered at the joints.

Assembled the robot in fusion and used plugin to export URDF

Manually exported inertia matrix from solidworks, provided by mechanical engineer but matrixes appear in wrong orientation and after multiple attempt, the URDF export did not yielded a usable URDF.

CAD matrix, validate accuracy and orientation with reference frame. Made tool to discover plausible inertia matrix from robot and validate against known one (since safe workspace is known, we get the robot to move with fixed base, leg in the air)

the full URDF procedure with screenshot should be done in appendix

The robot base come in at 5764g the hip come in at 3271g the cam come in at 66g the thigh come in at 483g the push rod come in at 71g the angle spring come in at 249g the shin come in at 324g the foot come in at 222g

3.4 Simulation

mujoco, URDF limitation with loop, GPU vs CPU, ...

3.5 Achievable gait: actuator and mass-distribution limits

The maximum running speed achievable by the robot was estimated independently of reinforcement learning using a first-principles analysis of the mechanical system. While RL demonstrates a lower bound on performance, this analysis provides an upper bound imposed by the robot's kinematics, dynamics, and actuator capabilities.

3.5.1 Method

The analysis was performed directly on the MuJoCo model (`training/model/dash01.xml`), ensuring consistency with the model used for policy training. Candidate gaits were parameterized by duty factor D , contact length L_c , ride height h , and cadence.

For each candidate gait, the corresponding stride length, flight time, swing duration, and ground reaction forces were computed using a simplified running model. Joint torques and velocities were then obtained through the leg Jacobian throughout the gait cycle.

A gait was considered feasible only if all joints remained within the admissible 4-bar linkage assembly region while respecting actuator torque, speed, and power limits. The parameter space was exhaustively explored to determine the maximum feasible running speed.

The implementation was validated against MuJoCo. Static equilibrium predictions agreed within 1%, while the analytical Jacobian matched `mj_jacGeom` with an error below 10^{-10} .

3.5.2 Results

Regime	Top speed	Limiting constraint
Continuous operation	1.8 m/s	Continuous actuator torque (55 N·m)
Cadence ≤ 6 steps/s	3.7 m/s	Cam actuator peak power
Cadence ≤ 8 steps/s	4.4 m/s	Cam actuator peak power
Unconstrained optimum	6.9 m/s	Cam actuator peak power

Table 3.1: Maximum running speed predicted from the robot model assuming a 350 N·m/rad ankle stiffness. Only the first regime is thermally sustainable; higher speeds correspond to burst operation.

At the practical optimum (≤ 6 steps/s), the predicted gait has a duty factor of approximately 0.45, a stride length of 1.1–1.3 m, a contact length of 0.25–0.30 m, and peak vertical ground reaction forces of approximately 3.5 body weights (440 N), with ground contact durations between 36 and 81 ms.

The predicted continuous-speed limit (1.8 m/s) is lower than the 2.6 m/s achieved by the RL policy, indicating that the learned controller relies on burst actuator capability. Consequently, motor temperature must be experimentally validated before sustained hardware operation.

The unconstrained optimum occurs at 12.4 steps/s, corresponding to the same high-cadence solution repeatedly discovered by reinforcement learning. This agreement confirms that the high-cadence behavior originates from the system dynamics rather than from a simulator artifact.

3.5.3 Limiting factors

Maximum speed is primarily limited by swing-leg actuator power rather than by kinematic reach.

The dominant constraint is the cam actuator during swing, which requires approximately 878 W (64 N·m at 13.7 rad/s), close to its theoretical maximum output of 935 W. Consequently, increasing actuator no-load speed would improve achievable running speed more effectively than increasing peak torque.

Stance loading represents the second limiting factor. Peak ground reaction forces require approximately 167 N·m of the available 170 N·m, leaving minimal torque margin.

Kinematic reach is not the limiting factor. Although the leg can theoretically achieve a fore-aft workspace of 1.06 m, only 0.20–0.35 m is practically usable. Near the edge of the assembly region, the transmission ratio of the 4-bar mechanism deteriorates, causing joint velocities to increase rapidly and reducing achievable running speed.

Introducing a flight phase substantially improves performance. Reducing the duty factor from 0.85 to 0.45 increases the predicted speed from 4.6 to 6.9 m/s while requiring only a 1.2 cm vertical center-of-mass excursion. Lower duty factors require ground reaction forces exceeding 4.5 body weights and are therefore infeasible.

3.5.4 Sensitivity to mass distribution

Added mass location	dv/dm
Uniform scaling	-0.14 (m/s)/kg
Torso payload	-0.11 (m/s)/kg
Entire leg	-0.43 (m/s)/kg
Distal leg only	-0.93 (m/s)/kg

Table 3.2: Sensitivity of predicted maximum speed to additional mass.

Running speed is considerably more sensitive to distal leg mass than to torso mass. Additional mass located in the feet, shank, or distal linkage reduces performance by nearly an order of magnitude more than an equivalent torso payload.

For the current design, removing 0.22 kg from each distal leg increases predicted top speed by approximately 0.5 m/s, whereas adding 5 kg to the torso produces a comparable reduction. This validate the heuristic used in the design of the leg that inertia should be minimized.

3.5.5 Ankle stiffness

A static load analysis independently reproduced the ankle stiffness threshold previously observed during reinforcement learning.

The passive ankle spring (28.65 N m/rad) supports only approximately half body weight before collapsing due to positive mechanical feedback, despite the actuators operating well below their torque limits. Increasing stiffness progressively improves load capacity, with approximately 350 N m/rad required to support two body weights and maintain stable stance.

The agreement between the analytical load model and the reinforcement learning experiments provides independent validation of this stiffness threshold.

3.5.6 Limitations

The analysis provides an optimistic upper bound on performance. It neglects torso attitude stabilization, lateral dynamics, impact losses, control bandwidth limitations, and the coupled actuator torque-speed envelope. In addition, CAD inertial properties remain approximate, although the sensitivity analysis indicates that distal link inertia strongly influences the predicted performance. The analysis should therefore be repeated once experimentally identified inertial parameters become available. UPDATE HERE ONCE I VALIDATED THE INNERTIA MATRIX OF EVERY SEGMENT !

3.6 Toy problem / RL Warmup

little inverted pendulum setup with one joint, one motor, one BBB running linux and sending command over CAN.

3.7 RL policy design and validation

The control policy went through three distinct action-space designs over the project. This section documents all three, the reflex structure that ended up wrapped around the final one, and the results obtained so far. Throughout, the learning algorithm itself was held fixed : PPO from

Stable-Baselines3, an actor-critic MLP, trained on CPU-parallel MuJoCo environments. So the comparisons are about the *action representation*, not about the optimiser.

3.7.1 Attempt 1: direct joint torque

The first framework commanded joint torque directly: the policy output was mapped to motor setpoints through a fixed torque scale on the actuators. This work was attempted with a simplified bipedal robot as the one developed in this thesis was still in development. This is the textbook formulation and the one most legged-locomotion papers use in simulation.

But, it was abandoned for two reasons:

It did not converge. The training history from that period is an unbroken sequence of reward-shaping attempts : raising the fall penalty because the robot learned to belly-flop, adjusting the alive bonus because it learned to dawdle, re-tuning the feet-air-time thresholds because it learned to vibrate in place. After a few iteration of this whicked whack-a-mole exploit patching, It was realised that with a raw torque action space the policy has to synthesise, from scratch and at every control step, both the gravity compensation that keeps the robot standing *and* the periodic structure that makes it walk; the reward landscape in between is dominated by ways of falling over that tricked the reward into thinking it was doing well.

More decisively, it was not deployable. The CubeMars drives expose a servo mode with an internal position/velocity loop running on the drive itself; commanding torque means bypassing that loop and streaming current setpoints over CAN from Python on a Raspberry Pi. I managed to push the measured CAN loop rate on our hardware to 200Hz but with millisecond-scale jitter and occasional missed deadlines, which is one to two orders of magnitude short of what torque-level biped control normally assumes; Furthermore, the CAN bus bandwidth would not have been sufficient, to stream data faster. A missed deadline in torque mode means the drive holds the last commanded torque blindly, which lead to constant acceleration.

The two failures point the same way: the policy class that most needs high control bandwidth and an accurate actuator model is exactly the one for which we have neither. Torque control was therefore dropped.

3.7.2 Attempt 2: joint position targets (PD)

The second design made the action six *position* targets, one per actuated joint, at 50 Hz, with the knee and ankle following passively through the parallel linkage and the ankle spring. In simulation these are MuJoCo `<position>` actuators with gains and force ranges taken from the datasheets of the cubemars actuator (**kp** 120, **kv** 4, 72 N·m at the hip-roll; **kp** 200, **kv** 5, 170 N·m at cam and thigh). On hardware this maps one-to-one onto what the drives actually accept.

What worked. Standing and balancing converged after a fair ammount of effort and reward shapping. After switching to a point-foot contact model only a 2.5 cm sphere at each toe collides, so the policy cannot rest a shin on the ground and damage the real robot, the policy held full-length episodes on two toe points. This also surfaced a floor-penetration exploit: with only the toe sphere colliding, the non-colliding foot and shin meshes could pass *through* the floor, and the first apparently-perfect balancing policy was largely exploiting that by slipping trough a stable equilibrium by lowering its CoM below the floor. It was fixed by terminating on both an upper-hemisphere toe contact (the sphere sank) and any foot/shin vertex below the floor plane.

What failed: skating. Walking did not. After 32 million steps the policy reached a perfectly healthy-looking reward curve and was *sliding*, not stepping. Both feet grounded on 57–66 % of

steps, duty factor 0.74–0.83, median “swing” duration of one control step (contact chatter at 5–10 touchdowns per second), in-contact toe slip of 0.11–0.24 m/s accounting for essentially all of its translation and a mean forward velocity of -0.09 m/s under a $+0.5$ m/s command.

Three separate causes were found and fixed: there was no price on foot motion while in contact (a slip penalty was added, with a grounded predicate that survives micro-hop and ghost-drag dodges); the air-time term could only ever punish stepping, since credit was paid at touchdown and never lifting scored zero (it was made one-sided and capped, and paired with a per-foot stance-time cap and a swing-clearance term to give PPO an immediate gradient toward lifting a foot); and the contact physics silently made skating free, because `condim=3` disables torsional and rolling friction, turning the toe spheres into ball casters. Two reward-plumbing bugs compounded it: observation normalisation was also normalising the *reward*, dividing the carefully balanced raw scale by a return standard deviation of ≈ 110 so that a 200-point fall penalty reached the optimiser as -1.8 ; and the entropy coefficient on a clipped Gaussian policy was effectively free entropy, so the action standard deviation ratcheted up until training behaviour and deterministic evaluation were different controllers.

The lasting methodological lesson is the first one: **the system will happily exploit any loophole in the reward function.**

Why we moved on. Even after the fixes, the fundamental problem remained: a per-step position policy must rediscover the periodicity of a gait from scratch at every control step. Nothing in the representation says that locomotion is cyclic. Compounding this, I updated the the six motor masses (7.1 kg, more than half the robot) and re-settling the stance keyframe under gravity moved the plant enough that the earlier standing policy no longer transferred, and the walking policy that had been slowly improving stopped improving at all. Rather than re-tune the same representation against a heavier plant, the representation was changed.

3.7.3 Attempt 3: a central pattern generator tuned by the policy

The third design puts the periodicity into the action space itself. Instead of emitting joint targets, the policy emits the *coefficients of a Fourier series* describing one gait cycle, and the environment reconstructs the joint targets from those coefficients at the current gait phase. This is a central pattern generator whose shape is learned rather than hand-authored: the network no longer has to invent periodicity, only to choose which periodic signal it wants. The work of the biorob was discovered after this idea emerged.

Formulation. For the two sagittal joints of the left leg (cam and thigh, the pair that drives the parallel 4-bar and therefore propulsion), the deviation from the nominal stance posture at gait phase φ is

$$\Delta q_j(\varphi) = A_j \left[w_0 a_0 + \sum_{k=1}^N w_k (a_k \cos k\varphi + b_k \sin k\varphi) \right], \quad w_k \propto \frac{1}{\sqrt{k}}, \quad (3.1)$$

with $N = 3$ harmonics, so seven coefficients per joint. The harmonic weights w_k are normalised to sum to one, which bounds the waveform, and the $1/\sqrt{k}$ schedule was chosen deliberately over the more obvious $1/k$: a $1/k$ schedule biases the series so strongly toward its fundamental that every gait it can express is near-sinusoidal, i.e. walking, whereas the push-off of a running stride is impulsive and needs high-harmonic authority. The result is clipped to $\pm A_j$ of the nominal posture a *relative* bound, because the nominal is always a validated, assemblable posture and the coupled 4-bar is only assemblable over a diagonal band of the joint box.

The phase φ advances at a frequency that is itself part of the action, so the policy chooses its own cadence. The right leg is not emitted at all: it is constructed as the mirror of the left, half a cycle out of phase, $\Delta q_{R,j}(\varphi) = -\Delta q_{L,j}(\varphi + \pi)$. This halves the action dimension, guarantees an alternating gait by construction, and matches the convention already used by the hand-authored gait demo. It has one consequence worth flagging, because it cost us time later: a mirror-symmetric gait can only travel in a straight line, so steering had to be added afterwards as two explicit asymmetry channels (differential stride amplitude and differential stance width) rather than emerging on its own.

First version: per-cycle updates, and why it was not enough. The initial implementation made one environment step equal one full gait cycle: the policy chose a gait, the environment ran it to completion (roughly 30 decisions per episode instead of 1000), and the macro reward was the mean of the per-step rewards. This was immediately better than the per-step PD policy — a hand-crafted elliptical foot path pushed through the same reconstruction walked at 0.357 m/s open-loop, versus the PD policy’s 0.01 m/s, and the first trained policy walked properly at ≈ 0.5 m/s with no skating signature at all. The structural prior worked exactly as intended.

But it was slow, and it fell over as soon as the pitch degree of freedom was released. A literature review at that point returned an unambiguous verdict: **no published system achieves running with per-cycle-only parameter updates and no intra-cycle feedback**. Every successful periodic parameterisation keeps a per-step channel — PMTG modulates the trajectory generator and adds per-step residual corrections; CPG-RL re-selects the oscillator parameters at 100 Hz. The same review found a genuine semi-Markov bug in our formulation: taking the *mean* of the per-step rewards over a macro-step, with a fixed discount per macro-step, makes the total return scale with cadence and makes the effective horizon vary between 4.8 s and 28 s depending on the gait frequency the policy happened to choose. It also flagged two things the observation was missing — base velocity (the quantity the reward maximises was not observable at all) and distance-to-the-line (so the value function was partially observed and braking was unplannable).

Final version: per-step re-parameterisation plus residuals. The current stack keeps the Fourier gait but re-emits the entire specification at *every* control step, and adds a per-step correction channel on top. The 24-dimensional action is:

dims	channel	role
14	Fourier coefficients	cam + thigh gait shape, Equation (3.1); mirrored to the right leg
1	gait frequency	cadence, 0.5–4.0 Hz; the phase integrates it
3	abduction reflex gains	k_p , k_d , bias of the lateral balance reflex
6	per-step residuals	direct corrections on the six PD targets, ± 0.08 rad

Table 3.3: The per-step action. The first 18 dimensions are the gait *specification*; the last six are the fast-feedback channel.

Rewriting the gait specification mid-cycle is legal but priced: a penalty proportional to $\sin^2(\varphi/2)$ bills changes to the specification, so a rewrite is free exactly at the cycle boundary and most expensive mid-stride. The residuals are deliberately *never* billed by it, because they exist to be per-step — a one-off stumble correction should not have to pay for rewriting the whole gait. This gives a clean separation: the Fourier channel is the gait, the residual channel is the correction, and a separate magnitude penalty keeps the residual from quietly becoming the controller.

The observation is 55 values stacked over 5 frames (275 total): motor position, velocity and torque; IMU gravity direction and gyro; base velocity; $\sin \varphi$ and $\cos \varphi$; a task channel carrying the run/stop flag and the distance remaining; and the previous action.

Reward. The sprint objective is a single dash — stand at the line, run, stop. The core insight, worth stating because it is not obvious, is that **a per-step speed reward cannot price time**: integrating $w v_x$ over a fixed course gives $w \cdot \text{distance}$ regardless of pace, identical for a fast run and a slow one. What prices time is a *constant per-step clock cost*, which integrates to $-w_t T$; and correspondingly the alive bonus must be exactly zero, because paying per second of existence rewards dawdling. The break-even pace is the ratio of the two, 0.25 m/s.

Around that core sit the anti-skate terms inherited from the PD era, a Siekmann-style phase-gated contact schedule (each foot pays for ground contact during its expected swing window, using the action’s own gait clock; as the stance ratio ramps from 0.65 down to 0.42 the two swing windows begin to overlap, so a flight phase is explicitly demanded — this is the one term that asks for running rather than fast walking), and efficiency terms on torque, motor velocity and positive mechanical power, ramped in over 60 million steps so they cannot smother gait emergence early.

Four reward exploits were found by adversarial review of the reward code and fixed before the long runs: a stop-phase income that was net-positive per step and therefore farmable forever by twitching the hold timer; a one-sided velocity clip that made shuttling back and forth in front of the finish line strictly better than crossing it; and a cold-start suicide, where the sum of always-on penalties for a standing robot (≈ -2.2 per step) made diving at -100 value-optimal for any policy that could not yet locomote — fixed by flooring the per-step total, not just the individual terms.

Curriculum over base degrees of freedom. Rather than ask a fresh policy to solve balance and locomotion simultaneously, the base is progressively released. The free joint was replaced in the model by six explicit scalar joints (x, y, z , roll, pitch, yaw) plus six equality constraints that can be activated individually, so any subset of the base degrees of freedom can be rigidly railed at reset. Milestone **m1** frees only forward translation (with height railed at a randomised ride height drawn from a measured lookup table), **m2** adds height, **m3** pitch, **m4** lateral translation, **m5** roll, **m6** is the real plant. Rigid equality constraints were chosen over clamping the state after each step precisely because they produce the correct reaction wrench *during* the step, so the leg torques stay faithful. Critically, the observation and action dimensions do not change across milestones, so every stage warm-starts from the previous one’s checkpoint.

3.7.4 Reflexes: what is learned and what is hard-wired

Not everything in the controller is a Fourier series, and the split is deliberate. A Fourier series over gait phase is by construction *feedforward*: it is a function of where we are in the cycle, not of what the robot is doing. Balance is not a periodic problem, it is a feedback problem, and encoding it as a phase-indexed trajectory would make it blind. Three feedback channels therefore sit outside the CPG.

Lateral (abduction) reflex — learned gains, fixed structure. The hip-roll joints are not driven by the gait generator at all. They are driven by a linear reflex on base roll,

$$u = k_p \phi_{\text{roll}} + k_d \dot{\phi}_{\text{roll}} + b, \quad q_{\text{hip roll},L} = q_L^{\text{nom}} + u, \quad q_{\text{hip roll},R} = q_R^{\text{nom}} - u, \quad (3.2)$$

evaluated at every control step. Because the left and right roll axes are mirrored, opposite signs in joint space move both feet the same physical way, which is the lateral centre-of-mass shift

we want. The three gains (k_p, k_d, b) are *emitted by the policy*, so the controller is learned — but only three numbers of it are, rather than a free trajectory. This was the design decision that made the original per-cycle formulation tractable at all (a feedforward CPG for propulsion plus a feedback reflex for balance, instead of a per-step policy with free abduction), and it was kept in the per-step version because it works. Nothing assumes which sign stabilises; the policy discovers that. The reflex is necessarily inert while roll is railed (milestones m1–m3, where the gains are unidentifiable) and only becomes live at m5 — expected behaviour, not a bug, and worth knowing before reading those training curves.

Sagittal (pitch) reflex — hand-tuned, not learned. A fixed PD on base pitch and pitch rate drives a *symmetric* fore/aft thigh offset,

$$u_p = -\text{clip}(k_p^\theta g_x + k_d^\theta \dot{\theta} + b^\theta, \pm u_{\max}), \quad q_{\text{thigh},L} += u_p, \quad q_{\text{thigh},R} -= u_p, \quad (3.3)$$

where $g_x \approx \sin \theta$ is the fore/aft component of the measured gravity direction. Again the thigh axes are mirrored, so the opposite signs move both feet the *same* physical direction: when the robot pitches nose-down, both feet shift forward, under the falling centre of mass. This is capture-point control in its crudest form, and the gain scales follow that reading ($k_p \approx h_{\text{CoM}}/\ell \approx 1$, $k_d \approx \sqrt{h/g} \approx 0.3$).

The reason this one is *not* learned is worth spelling out, because it is the opposite choice from the abduction reflex. It was introduced when the pitch-free milestone was collapsing in 1.5 s, and a learned gain has no useful gradient in that regime: the policy has to survive long enough to experience the correction before it can learn to tune it, and it was not surviving. A hand-tuned prior breaks that circularity. The gains were set by a grid search on the pitch-free plant: $k_d \geq 0.3$ oscillates through the one-step action delay and the target smoothing filter, and an authority clip above 0.35 rad starts fighting the gait itself, which fixes $k_p = 2.0$, $k_d = 0.2$, clip 0.25 rad. Measured on a pitch-kick arrest test, the reflex reduces the final pitch excursion by a factor of 2.6–6.7 in both directions, and lifts early-retraining survival from about 18 to about 80 control steps.

That number is the point: the reflex is a **stabilising prior, not a controller**. The plant with height free cannot stand at zero action at all — holding a ride height requires an active gait, so it collapses in ≈ 1.3 s no matter what the pitch does. The reflex does not fix that; it buys the policy enough surviving episodes to find a pitch-neutral gait itself. An early acceptance gate of “zero action stands for 20 seconds” was simply wrong for this plant and was replaced by the kick-arrest test.

Residuals — the fast channel. The six per-step residuals are the third feedback path, and the one the literature is most emphatic about: they are what lets the controller respond to something unmodelled within a single control step without rewriting the gait. They are kept small (0.08 rad, raised to 0.20 rad once the control rate moved to 200 Hz) and are magnitude-penalised, so the Fourier prior stays dominant.

All three are implemented in pure NumPy, in the same module that reconstructs the gait, with no dependency on the simulator. That is not incidental — it is what allows the identical reflex code to run on the Raspberry Pi against a real IMU, rather than being re-implemented and re-tuned at deployment time.

A negative result: the emulated ankle reflex. One more reflex was built and rejected. Since the ankles are passive springs with no actuator, we implemented a fixed PD that applies a pitch-restoring *torque* at the ankle joints, gated on that foot being grounded — an emulated

actuated ankle, and a shippable controller if a real ankle motor were ever added. At 40 N·m of authority it *destabilised* the running gait outright (episodes collapsed to 28 steps): a phase-blind torque fights the gait it is trying to stabilise. At a gentle 12 N·m it neither destabilised nor solved anything, giving a small, non-solving lift. The negative result was still informative — it was evidence that ankle authority was the right lever, which is what motivated the passive-stiffness experiment described next.

3.7.5 The pitch-balance wall, and how it was broken

Everything through milestone m2 (forward translation and height free, attitude railed) trained well. Releasing pitch at m3 produced the hardest failure of the project: episodes plateaued at roughly one second — the passive fall time — and stayed there. Two full sessions of work went into it, and it is worth recording what was ruled out, because the null results are the useful part.

What was tried	Outcome
Reward/curriculum easing (7 variants: no flight demand, efficiency off, low speed cap, more authority, height shaping, competence-gated hardening)	All plateaued at the same episode length. The reward was not the bottleneck.
Decaying external pitch assist (a spring-damper on the base pitch joint, faded to zero)	<i>Crutch dependence.</i> In-training episode length looked excellent; with the assist forced to zero at evaluation the policy was <i>worse</i> than the plateau, and drifting backwards.
Anti-crutch penalty on the assist torque the policy provokes	No improvement at low weight, worse at high weight.
Pitch “slow motion” via extra armature on the base pitch DOF, faded out	Marginal gains that did not survive the fade.
Raising the control rate 50–200 Hz and increasing residual authority (0.2–0.4) and filter bandwidth	All plateaued at ≈ 1 s. Since the learned residual gives <i>more</i> foot-placement authority than any fixed heuristic could, this implicated the plant rather than the control law.
Emulated actuated ankle (fixed torque reflex)	Strong version destabilised; gentle version marginal.
Lowering the centre of mass by 6–12 cm	Slightly <i>worse</i> .

Table 3.4: The pitch-balance failure: everything that did not work.

Two methodological lessons came out of this table. First, **any training aid that holds the robot up will be leaned on**, and the in-training curve will look wonderful while the unaided policy gets worse; only an evaluation with the aid forced to zero tells the truth. We were misled by this three times before adopting the assist-off evaluation as protocol. Second, when every software lever saturates at the same value, the constraint is not in the software.

The diagnostic that cracked it. Rather than try a ninth lever, we instrumented the failure: a tool that rolls out a policy and plots, per foot, the fore/aft offset between the toe at touchdown and the whole-robot centre of mass. The result was unambiguous. The base pitch ramps monotonically forward into a pure topple, and meanwhile the feet land 8–9 cm *behind* the centre of mass — only 11 % of touchdowns landed ahead of it, and they fell progressively further behind as the body ran away forward. The policy had a perfectly serviceable forward-running gait and had never learned the *capture step*: plant the foot ahead of the centre of mass to catch a forward fall. It was, quite literally, running out from under itself.

The fix was a plant change. The candidate lever this pointed to was the passive ankle. Stiffening the ankle spring by roughly a factor of twelve (28.65–350 N·m/rad) gives the passive foot the stance pitch-restoring authority the plant lacked. The threshold is clean: $k \leq 200$ barely helps, $k \geq 350$ breaks out of the plateau and climbs steeply. Post-fix, the same diagnostic reports 67 % of touchdowns landing ahead of the centre of mass and the base pitch pinned near zero.

One implementation detail is essential and cost us a wasted run. The standing ankle sits at $\approx \pm 0.25$ rad while the spring reference is at ± 0.7 rad, so the spring carries ≈ 12.8 N·m of *preload* at stance. Naively raising the stiffness balloons that preload, the foot slams to its spring reference and the robot flips over. The reference has to be shifted simultaneously so that $k(q_{\text{stand}} - q_{\text{ref}})$ is preserved: only the restoring *gain* rises, the posture is unchanged.

This result also *overturned* our own earlier recommendation. After the emulated-ankle experiments we had concluded that the robot needed a policy-controlled actuated ankle — a motor, two more action dimensions, and a full retrain of the milestone ladder. The stiffness experiment shows a passive spring is sufficient, which is dramatically cheaper. The open question is now mechanical rather than algorithmic: whether the real ankle and foot structure can be built that stiff. And, as noted in Section 3.5, the static load analysis independently arrives at the same 350 N·m/rad threshold, which is reassuring.

3.7.6 The cadence problem and a cautionary bug

With balance solved, the remaining pathology was cadence: the policies stepped far too fast, on the order of 11 Hz, pattering rather than striding. Several external limits were tried — hard velocity caps, acceleration caps, a cadence penalty — and all of them collapsed the warm-started policy instead of slowing it.

Instrumenting the trained policy’s own action channels found why. When the control rate was raised to 200 Hz, the learnable gait-frequency range had been widened to 0.5–50 Hz on the reasoning that it would then be “non-binding”. But the map from the raw action to frequency is linear over that range, so a *neutral* network output (≈ 0) means 25 Hz. The policy simply left that output near zero and inherited a 24 Hz gait clock — the commanded thigh trajectory reversing sign 35 times a second at ± 0.45 rad, a signal that would destroy the real actuators. Every external limit had been fighting a broken internal default. Remapping the range to 0.5–4.0 Hz, so that a neutral action means ≈ 2 Hz, produced the best controller of the entire project (Table 3.5).

The general lesson is worth carrying: **a parameter range widened to be “non-binding” can silently become the operating point**, because a network’s default output is not a neutral gait — it is whatever the middle of the range happens to decode to.

A residual pathology survives the fix. After the remap, the gait clock slowed to ≈ 1 Hz but the actual *footfall* rate stayed near 6 Hz: the policy steps through the fast residual channel rather than through the clock. A ring-down test showed the passive ankle mode sits at 1.0 Hz, not 6.3 Hz, so this is a forced oscillation, not a resonance — the policy pumping its legs to

balance a top-heavy robot at 200 Hz. Neither a residual-rate penalty, a torque-budget curriculum, reducing the pitch-reflex derivative gain, low-pass filtering the reflex input, nor heavy ankle damping removed it, and warm-started runs are all trapped in the fast-stepping basin. The honest conclusion is that it needs a from-scratch retrain with cadence pressure present from step zero, or a plant change (a lower centre of mass); on hardware, the real mechanical damping of the ankle would absorb some of it.

3.7.7 Results

Milestone	Free base DOF	Steps	Survival	Outcome
m2	x, z	180 M	53 s	runs; the reliable working regime
m3	+ pitch	600 M	39 s	completes the dash and stops, 2.6 m/s
m4	+ y	600 M	38 s	completes the dash and stops
m5	+ roll	600 M	26 s	high variance — the weak link
m6	all six	600 M	60 s	never falls; runs through the line
m7	+ pitch (cadence fix)	163 M	37 s	best controller: 2.71 m/s, very low variance

Table 3.5: Deterministic (greedy) evaluation of the milestone ladder, each stage warm-started from the previous. m7 is not a new degree of freedom but the m3 plant retrained after the gait-frequency fix of Section 3.7.6. Survival is capped at 60 s by the episode limit, so m6 is at the ceiling.

The headline result is that **the fully free 6-DOF plant runs and does not fall** — 60 seconds of simulated running, at the episode limit, with zero variance across evaluation episodes, at 2.6 m/s. Two caveats belong next to it: m6 balances but runs *through* the finish line without executing the stop (a task-completion defect, not a balance one), and m5 still topples on a fraction of episodes and needs more training. Against the capability bound of Section 3.5, 2.6 m/s is about 70 % of the credible-cadence ceiling and already above the thermally sustainable figure.

Cost. Training ran on the EPFL Izar cluster. Throughput is ≈ 2900 environment-steps per second on a V100 node, which is $\approx 1.9\times$ the CPU-only rate on the same node — and, worth recording, not because the network is large. Physics is always on the CPU (there is no GPU physics backend in this stack); putting the tiny policy network on the GPU simply stops PyTorch from stealing cores from the parallel MuJoCo workers. At that rate a 600 M-step milestone is a multi-day job, which is why the warm-start chain across milestones matters as much as it does.

3.7.8 Validation status and known gaps

Stating this plainly matters more than the results above, because the gap between them is the work that remains.

Sim-to-sim is exercised continuously by the milestone ladder: each stage is a different plant (different constrained dynamics) and every stage loads the previous stage’s weights, so a policy that only worked by exploiting a particular rail fails visibly at the next rung. The ankle-stiffness and centre-of-mass experiments are the same mechanism used deliberately as a plant-sensitivity probe.

Sim-to-real has not been attempted, and the sprint policies in Table 3.5 are not deployable as they stand. They observe two privileged quantities that do not exist on the robot: the base linear velocity (which would require a state estimator we have not built) and the distance remaining

to the finish line. A separate policy lineage was therefore started for a drivable demonstrator, which removes both — velocity has to be inferred from a 200 ms strided observation history instead — and adds per-episode plant randomisation (mass, inertia, friction, servo gains, joint damping, ankle spring stiffness, gravity tilt, actuation delay), a sensor-noise model including the accelerometer leakage that makes a real IMU’s “down” swing by several degrees during push-off, and the steering channels the mirror-symmetric gait lacks. That lineage cannot warm-start from the sprint milestones — the observation and action dimensions both changed — so it is a separate training cost, currently in progress.

The remaining known gaps, recorded deliberately rather than discovered later: there is no policy export path and no inference loop on the Pi, so nothing has run on hardware yet; the actuator model gives each motor an independent torque limit and speed limit rather than a coupled torque–speed envelope, which makes the simulation optimistic exactly in the high-speed regime the project cares about; and the CAD link inertials are still placeholders, which is why the domain-randomisation range on inertia is as wide as $\pm 25\%$ and why the capability analysis of Section 3.5 should be re-run once measured values land. First hardware bring-up is planned with the base fixed and the legs in the air, which validates wiring, direction conventions, joint limits and gait shape — but, with no ground contact, tells us nothing about balance or transfer.

3.8 deployment

3.8.1 linux and RT

3.8.2 CAN

3.8.3 motor controller

3.8.4 hardware validation

webUI with simple path, safe limits, ...

3.9 Hardware experiments

4 Results

4.1 Hardware selection

show motor, battery pack, ...

4.2 RL experiments

show all the practical result we learned.

5 Discussion

5.1 Successes

What worked? pipeline modelling standing gait formulation

5.2 Failures

What did not work? direct torque RL fully autonomous running

5.3 Lessons learned

Simulation fidelity matters. Action-space design matters. Hardware integration dominates development time. Progressive validation reduces debugging complexity.

6 Conclusion and Future Work

6.1 Summary of Findings

6.2 Challenges and Limitations

6.3 Suggestions for Future Research

7 Appendix

7.1 logbook

- 2025-02-20** Creation of this report, rough layout of the idea in chapter
- 2025-02-25** Longitudinal kinematic model, research about markov modelling and cycle
- 2025-02-26** dynamic simulation creation, implemented a basic simulation in pybullet with keyboard navigation and parametric surface + wheel rolling down
- 2025-02-27** refining efficiency to take into account whole cycle, reading paper
- 2025-02-28** Writing Idea about improving efficiency through identified parameters
- 2025-03-04** Adding graph about car occupancy and real world driving needs + reading paper
- 2025-03-06** Brainstorming kinematic, reading PhD thesis on narrow track
- 2025-03-09** URDF is a PAIN to implement to due lack of tool, trying to define a kinematic model in python
- 2025-03-11** reworking part of chapter 2
- 2025-03-17** reading paper about cars and bike and limiting factor to adoption
- 2025-03-18** keep reading paper, also about narrow track vehicle
- 2025-03-20** trying to structure what I learned from the papers
- 2025-03-21** defining the know unknown about car adoption factor (human side)
- 2025-03-24** Clarifying the project goal
- 2025-03-27** draw figure, added citation
- 2025-03-28** Manual definition is an epic failure... Two way (createMultiBody -; Works but hard to read), Create body and add constraint don't work (rotation constraint crash the simulation.)
- 2025-04-09** Realized that createMultibody is more stable but has a "ghost" friction problem, Might as well use URDF then.
- 2025-04-10** learning URDF and how the origin is defined,
- 2025-05-25** redefined most variant as URDF and how to simulate them. Need spend time to implement basic controller like PID to show steady state stability and control margin
- 2025-05-28** Meeting, dry run presentation and shown current status.
- 2025-05-31** Report restructuring, writing
- 2025-06-02** Chapter 4, 5, 6 writing
- 2025-06-04** Getting simulation result, finishing first draft of report.
- 2025-06-08** Proof reading

7.2 URDF from CAD:step by step

recall limitation of URDF regarding parallel robot, tricks with closed loop and equality constraint, other format that are better suited but did not manage to export. add screenshot and show how to do it for the full robot.

References